

Alpha-GPT: Human-AI Interactive Alpha Mining for Quantitative Investment

Saizhuo Wang^{*†}

The Hong Kong University of Science
and Technology
Hong Kong SAR
swangeh@connect.ust.hk

Hang Yuan^{*†}

The Hong Kong University of Science
and Technology
Hong Kong SAR
hyuanak@connect.ust.hk

Leon Zhou[†]

Columbia University
USA
Leon.zhou@columbia.edu

Lionel M. Ni

The Hong Kong University of Science
and Technology (Guangzhou)
China
ni@ust.hk

Heung-Yeung Shum

IDEA Research
China
hshum@idea.edu.cn

Jian Guo[‡]

IDEA Research
China
guojian@idea.edu.cn

ABSTRACT

One of the most important tasks in quantitative investment research is mining new alphas (effective trading signals or factors). Traditional alpha mining methods, either hand-crafted factor synthesizing or algorithmic factor mining (e.g., search with genetic programming), have inherent limitations, especially in implementing the ideas of quants. In this work, we propose a new alpha mining paradigm by introducing human-AI interaction, and a novel prompt engineering algorithmic framework to implement this paradigm by leveraging the power of large language models. Moreover, we develop Alpha-GPT, a new interactive alpha mining system framework that provides a heuristic way to “understand” the ideas of quant researchers and outputs creative, insightful, and effective alphas. We demonstrate the effectiveness and advantage of Alpha-GPT via a number of alpha mining experiments.

1 INTRODUCTION

A trading alpha [1] is a financial signal or a function with predictive power over excess return or risk, and they are usually expressed via symbolic rules or formulas (machine learning alphas are getting more popular recently but they are not discussed in this work). Most research work in quantitative investment focuses on how to find good alphas. See [2] for a number of such formulaic alphas (e.g., $-\frac{\text{close}-\text{open}}{(\text{high}-\text{low})+0.001}$ computes the increase from open price to close price relative to the intraday volatility, and the negative sign indicates a potential mean-reversion effect).

Traditionally, alpha mining has two paradigms (Figure 1). The first paradigm relies on manual modeling. Quant researchers attempt to translate their ideas/intuitions about financial markets into formulaic alphas, test their effectiveness and significance through backtest experiments, and analyze the reasons of success and failure. Usually, this process is repeated for many rounds to improve the performance of alphas. The success of this paradigm depends heavily on the talent and expertise of individuals and suffers from the problems of inefficiency and labor cost. On the other hand, the second paradigm seeks alphas through search algorithms such

as genetic programming [3]. Since the search space, composed of all possible combinations for hundreds of operators and operands (features), is incredibly large, it is extremely compute-intensive to find satisfactory alphas during the alpha search process.

Both of these paradigms exhibit common shortcomings. Firstly, it is a difficult process to find a precise and concise formulaic expression that encapsulates one’s ideas about trading signals or observed trading opportunities and patterns. Examples include the formulaic representation of technical analysis patterns such as Ascending Triangles [4] and Elliott Wave Theory [5], which exist but are hard to discover. Secondly, understanding and interpreting a large number of alphas selected by search algorithms is especially time-consuming and labor-intensive. Lastly, it is unreasonable to expect creative and effective alphas to come from the stroke-of-genius by researchers or the brute-force search by algorithms, but rather, it often comes from a repeated process of experimentation-and-analysis. However, designing and modifying the parameters and search configurations of algorithmic alpha mining systems is usually a menial task for researchers.

To address these challenges, we propose the third alpha mining paradigm which enhances human-AI interaction to improve the effectiveness and efficiency of alpha research. Based on this new paradigm, we propose the architecture of an interactive alpha mining system, termed **Alpha-GPT**. **This system incorporates large language models (LLM) as a mediator between quantitative researchers and alpha search.** Alpha-GPT has three key advantages. First, it can interpret users’ **trading ideas and translate them into fitting expressions**. Secondly, Alpha-GPT can quickly summarize **top-performing alphas in natural language for ease of understanding**. And finally, the user can then suggest modifications to the alpha search which the model will automatically make to future rounds of alpha mining. This greatly simplifies the workflow (Figure 2) and allows the user to approach alpha mining from a high-level standpoint (in terms of abstract ideas).

Our contributions in this work can be summarized from these standpoints:

- We define a new paradigm for alpha mining utilizing human-AI interaction to improve the effectiveness of alpha research.

^{*}Both authors contributed equally to this research.

[†]Work done during internship at IDEA Research.

[‡]Corresponding author

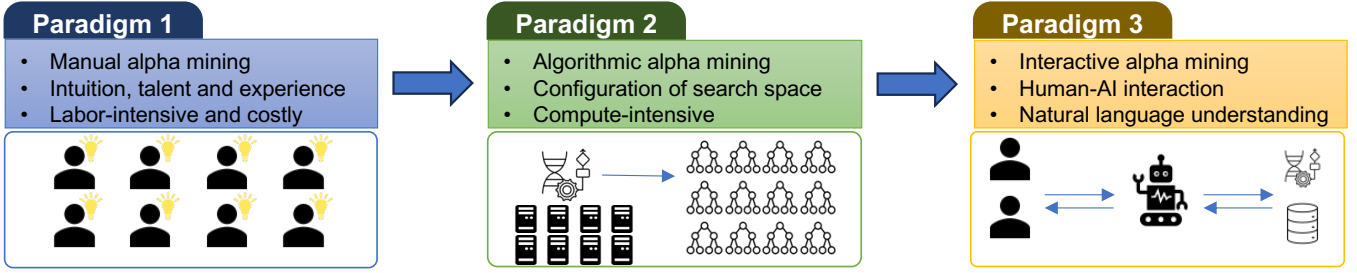


Figure 1: Evolution of alpha mining techniques.

- We propose AlphaBot, an algorithm with domain knowledge compilation and decompilation methods to employ the LLM as a mediator for human-AI interaction.
- We develop Alpha-GPT, a systematic framework incorporating AlphaBot to realize our proposed paradigm and a tool for quantitative researchers.

2 USER INTERFACE OF ALPHA-GPT

In Figure 3 we introduce the user interface (UI) of Alpha-GPT, which is composed of three main components: Session Manager, Dialog Box, and the Alpha Mining Dashboard.

Dialog Box: Users input their trading ideas and thoughts into the dialog box. In response, generated seed alphas, alpha searching progress, the final report of alpha mining, as well as the performance of the generated alphas, are all organized into system messages that provide comprehensive feedback. Users can then analyze the results and provide further direction for alpha mining, and this dialog continues until effective alphas are found.

Mining Session Manager: Alpha-GPT features a session-based user interface that stores past interaction history via the session manager. These sessions also serve to organize user-generated data, which can then be used to further enhance the system’s performance.

Alpha Mining Dashboard: On the right half, the dashboard is used to display and analyze alpha mining results. It provides users with more detailed descriptions of the session. **Experiment Monitor** displays the alpha mining experiment progress and current system load as well as all historical alphas generated during a session. If a specific alpha is selected, its performances are plotted and visualized on the **Analytic Panel**. The available plotting features include a fitness curve across generations for genetic programming, a backtest curve for a single alpha, and other peripheral analyses such as IC distribution and Signal decay. Furthermore, **Alpha Dashboard** includes one-click storage and deployment, enabling further application and analysis downstream.

3 ARCHITECTURE AND TECHNOLOGY CHALLENGES

Figure 4 shows the system framework we proposed for the interactive alpha mining paradigm by distilling and abstracting from the architecture design of Alpha-GPT. Since UI has been introduced in Section 2, we only introduce the design idea and technical challenge of other modules behind the system.

3.1 AlphaBot Layer

AlphaBot is the key layer of Alpha-GPT, as it plays a mediator role in human-AI interaction. Specifically, this layer consists of four functional modules: 1) **Knowledge compiler** automatically converts the intents/ideas/thoughts of quantitative researchers into domain-specific prompts and instructions for an LLM query; 2) **LLM** provides APIs or local deployment options for mainstream large language models such as GPT-4; 3) **Thought decompiler** translates the natural language output of an LLM into a configuration understandable by the algorithmic alpha mining layer; 4) **Knowledge Library** incorporates extra knowledge, information, literature and data about alpha mining to improve the performance and accuracy of LLMs.

3.1.1 Knowledge compiler. On domain-specific tasks like alpha mining, a lot of user requests will include terminology that can only be found in finance. Without further context, a traditional LLM would be unable to understand the meaning behind the inputs. This then introduces the need for the knowledge compiler module. Leveraging the in-context learning capability of LLM [10], the module enhances the original user request by providing additional context and clarifying keywords in prompts [11]. Specific phrases like “*you are a quant researcher developing formulaic alphas*” help narrow the scope of possible responses to answers more suited for factor mining.

Since the alpha expressions must also be in a certain format to be valid, there is a section in the prompt with each possible component of an expression along with an explanation of how it can be utilized (e.g. “*high_1D*”: *highest intraday price of stocks*). This addition helps the LLM correlate the user intentions with certain functions that can fulfill that goal, and helps ensure that the output will be valid. Because this terminology is applicable to the wider finance domain, this portion of the prompt is also used in the thought decompiler to prevent hallucinations. Such a module is required to allow users to make requests in natural language without the ambiguities involved.

3.1.2 Large Language Model. There are currently two main ways of utilizing LLMs, each with their own characteristics.

- **Online Request:** A swath of companies offer access to their pre-built large language models through APIs. These are products that charge a nominal amount per set amount of tokens and include GPT, Claude, and Bard, to name a few. One advantage of

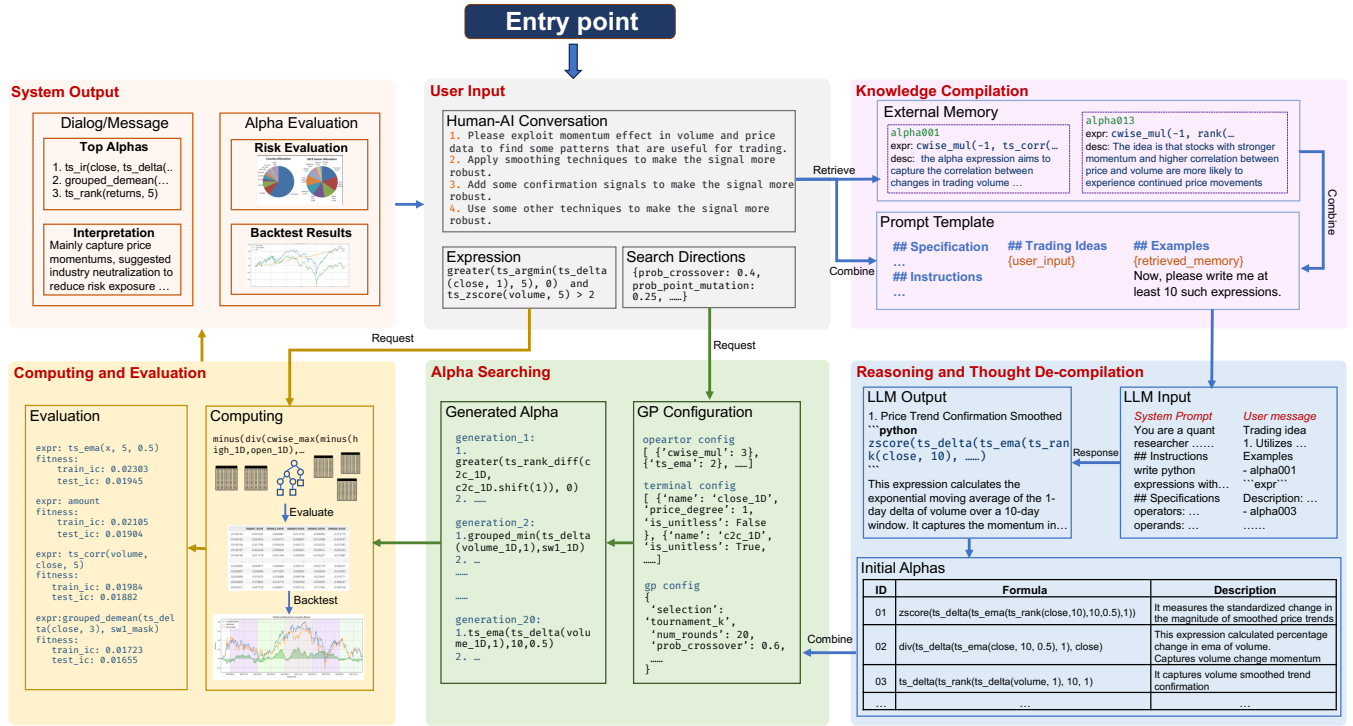


Figure 2: Alpha-GPT internal working pipeline: After a user inputs their ideas, the system goes into the knowledge compilation module. It uses external memory to pull similar examples, and combines them into the system prompt. The module passes everything to the LLM which creates valid alpha expressions and config files. These alphas are evaluated via Alpha Search, and results are presented to the user along with an interpretation provided by the Thoughts Decompiler.

APIs is that it allows for ease of use, and removes the need for computing power.

- **Local Deployment:** LLMs can also be developed locally from the ground up. This approach allows greater customization in how the model functions. They can be pre-trained on specific datasets to align them towards certain purposes. This involves domain fine-tuning by learning on relevant documents, and also reinforcement learning from human feedback [12]. The result is an LLM that is specifically built for a subset of tasks. However, the user then requires a large amount of computing power [13, 14] to train and update these models.

3.1.3 Thought Decompiler. There is a significant gap between LLM responses and the desired output structure in alpha mining. Specifically, the gaps can be summarized as challenges from the following perspectives:

- **Nature language to structured data:** We need to convert LLM response from natural language to structured data. For each LLM response message, we need to extract a list of expression blocks from their raw outputs. And each expression block follows a certain organization style such as its short name, expression, and a paragraph of natural language description.
- **Token size limit:** Since most state-of-the-art LLMs apply a Transformer-based architecture with self-attention mechanisms, limits on the input sequence length (number of tokens) is usually

Algorithm 1: LLM Reasoning and Thought Decompiling

Input: Input message history H , Expected number of alphas N , expected minimum number of alpha per round θ , retry limit τ

Output: List of generated alphas l

```

1  $l \leftarrow []$ ;
2 while  $\text{len}(l) < N$  do
3    $m \leftarrow \text{get\_llm\_response}(H)$ ;
4    $r \leftarrow \text{parse\_exprs}(m)$ ;
5   if  $\text{len}(r) < \theta$  then
6     continue;
7    $v, \tilde{v} \leftarrow \text{validate}(r)$ ;
8    $l \leftarrow l + v$ ;
9   for  $I$  in  $\{1.. \tau\}$  do
10     $\text{retry\_message} \leftarrow \text{make\_retry\_prompt}(H, \tilde{v})$ ;
11     $v, \tilde{v} \leftarrow \text{retry\_and\_validate}(\text{retry\_message})$ ;
12     $l \leftarrow l + v$ ;
13    $H \leftarrow \text{update\_prompt}(l)$ 
14 return  $l$ ;
```

a very common concern. In this way, both inputs and outputs of LLMs have an upper bound for their lengths. This may cause

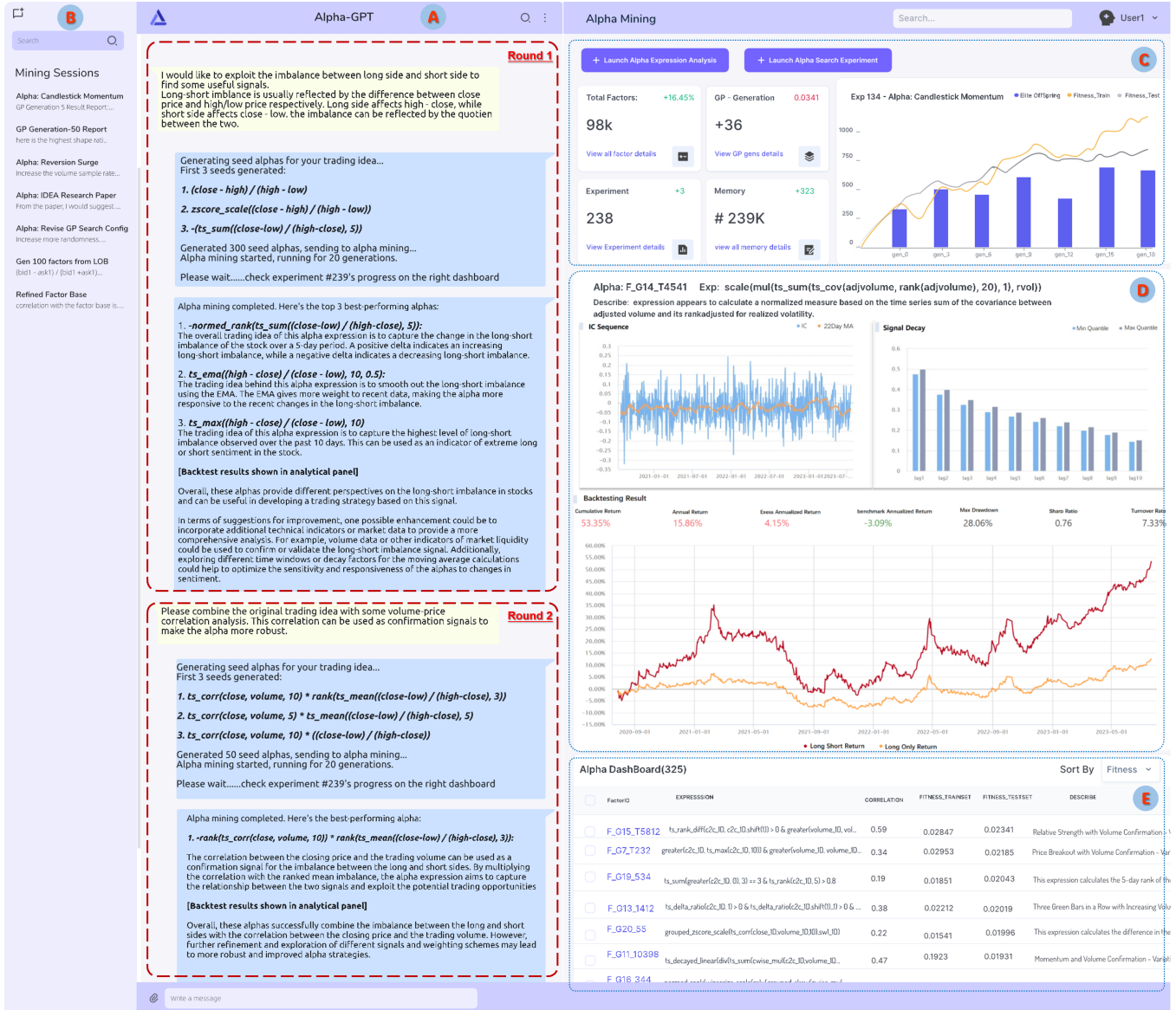


Figure 3: User interface of Alpha-GPT. Five modules of the UI are annotated as: (A) Dialog Box, (B) Mining Session Manager, (C) Experiment Monitor, (D) Analytic Panel, and (E) Alpha Dashboard

two issues: 1). We cannot keep sending full conversation history to LLMs since it will exceed the input token size limit. 2). The size of a single LLM response is limited, meaning that we can only get a limited number of expressions per message.

To address these issues, we propose an iterative LLM reasoning procedure (Algorithm 1). We apply a parser based on regular expressions to parse LLM outputs. For each LLM-generated alpha, we validate its correctness both syntactically and semantically. We adopt an abstract syntax tree parser for syntax checking, and for semantic correctness, we evaluate this expression with mock data and runtime context to capture any exceptions being thrown out.

In practice, the proportion of correct alphas might be low (only 4 out of 10 alphas generated per round), making the alpha generation process inefficient. Meanwhile, since the conversation history is appended to LLM inputs, these incorrect expressions may also affect the generation process that follows. Hence, we apply an iterative correcting procedure where we prompt the LLM to re-generate incorrect alphas. Moreover, to address the problem of exceeded token size limit, at each round we dynamically check if the token size limit is exceeded, and if so, the input message will be truncated and reorganized to reduce the number of tokens.

3.1.4 Knowledge Library. As mentioned in Section 3.1.1, few-shot in-context learning requires an external memory that supports

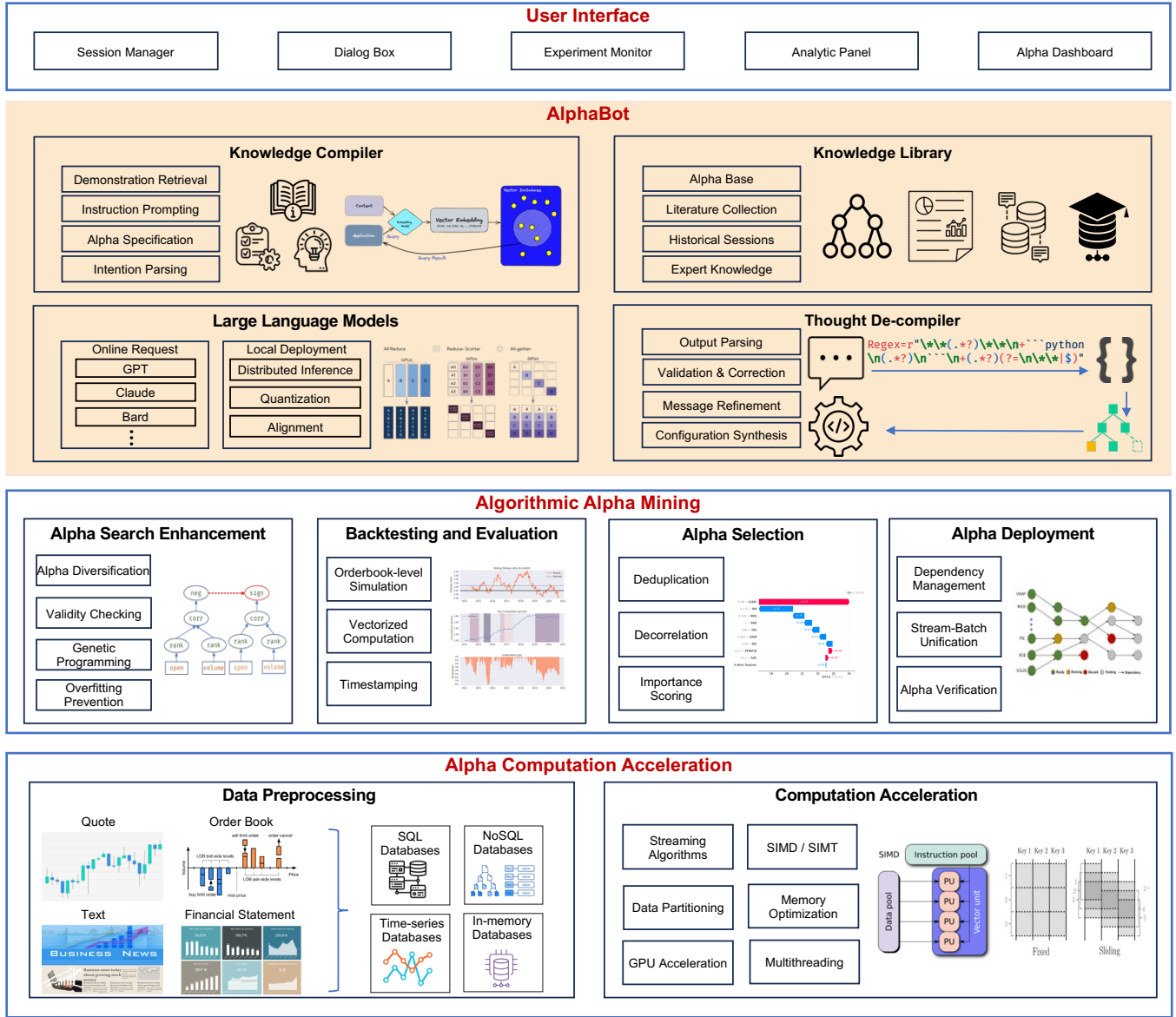


Figure 4: Alpha-GPT system architecture. Part of this figure is cited from [6–9].

efficient retrieval of demonstrations relevant to trading ideas. In Alpha-GPT, we design a protocol for organizing contents from across multiple sources. These range from an existing collection of alpha expressions to financial literature. When the user makes a request, the knowledge library encodes that query, and finds similar documents that it can incorporate into the prompt as examples.

3.2 Algorithmic Alpha Mining Layer

This layer serves a search enhancement function in Alpha-GPT. Specifically, it implements an algorithmic alpha mining workflow by taking seed alphas and improving them with the received search commands and configurations from AlphaBot, and pushes the most appropriate alphas back to AlphaBot. It consists of four modules:

the **algorithmic alpha search** module generates alpha candidates according to the commands from AlphaBot, qualified alphas are selected from these candidates using the **evaluation and back-testing** module, these alphas are further pruned according to a specific prediction target (e.g., contribution to the return of future 10 days) in the **alpha selection** module, and the final alphas are “one-click” deployed by the **alpha deployment** module to guarantee the smoothness and correctness of real-time computing during online trading.

3.2.1 Alpha Search Enhancement. The most popular alpha search algorithm used in industry is genetic programming (GP), which starts from a number of alpha seeds and iteratively selects formulae

alphas expressed by trees using random crossing and mutation subtrees according to the fitness of a scoring function. However, GP currently suffers from three problems: 1) **Overfitting**, which is extremely dangerous for quantitative trading. This can be mitigated by strategies such as out-of-sample evaluation incorporated in GP iterations, fit regularization to reduce function complexity, and early stopping of iterations. These methods help ensure alphas generalize well beyond training data, improving their reliability. 2) **Loss of diversity** in alphas may result in the aggregation and accumulation of investment risk and increase the uncertainty of returns. Alpha diversification can be realized by enforcing more constraints in GP’s iteration process, and it helps discover robust alpha factors that are resilient to changing market conditions. 3) **Invalid alphas** are easily generated by GP. For example, $\log(0)$ and $\sqrt{-5}$, or a sum of two values with incompatible units (e.g. volume + close). Incorporating a rule base encompassing mathematical rules, unit consistency rule and financial domain-specific rules could regulate alpha expression generation.

3.2.2 Evaluation and Backtesting. The most straightforward method of evaluating alpha is through backtesting, which reveals the alpha’s specific performance in an investment strategy. This process, however, introduces three significant challenges: 1) **Introduction of future information**: information from a further point in time when backtesting could have disastrous consequences on result accuracy. To mitigate this, Timestamping is used to assign a time label to each piece of input data. This technique allows the backtesting system to more accurately replicate market conditions and validate test results, thereby enhancing the reliability of alpha evaluation. 2) **Estimation of transaction costs**: conventional coarse-grained backtesting cannot accurately measure transaction costs, which is crucial for short-term alphas. To solve this, we conduct a simulation-based [15] backtest with trade matching rules using more detailed data, such as order book level data, for selected alphas. This enables us to model how transaction costs and market price impact alphas at a microstructure level. 3) **The computational cost**: the compute-power necessary for alpha mining is also significant, and we address this problem in the computation acceleration layer.

3.2.3 Alpha Selection. The Alpha Selection module furthers the selection process in the following ways: 1) **Deduplication and Decorrelation**: new alphas need to be distinct from existing ones, but calculating pair-wise similarity between large amounts of alphas can be time-consuming. Algorithms like KD-Tree, Locality-Sensitive Hashing (LSH) [16], or Approximate Nearest Neighbors (ANN) can swiftly determine each prospective alpha’s maximum correlation with others in the pool. 2) **Importance Scoring**: while an alpha’s IC score and backtest may reflect individual performance, in real scenarios, multiple alphas are combined together in investment strategies, and these metrics do not accurately reflect how they perform in a larger set. A group of alphas with low IC scores may outperform a subset composed of the highest-scoring alphas. Thus, importance scoring techniques such as Shapley Additive ex-Planations (SHAP) [17] and Local Interpretable Model-agnostic Explanations (LIME) [18] measure this contribution and provide a comprehensive understanding of alphas in conjunction with one another.

3.2.4 Alpha Deployment. In this module, three key aspects need to be properly managed to guarantee the smooth and correct real-time computation during online trading: 1) **Dependency Management**: This involves maintaining and supervising all alpha-data inter-dependencies to ensure sequential computation and traceability of issues. 2) **Stream-Batch Unification**: Inconsistencies between live trading and historical backtesting of alphas are unacceptable. By adopting the Kappa Architecture, a unified real-time processing layer is maintained. Thus, all data is processed in the same manner, eliminating inconsistencies between batch and stream processing during alpha generation. 3) **Automatic Alpha Verification**: This is employed to validate all system-maintained alphas, monitor data quality, and identify discrepancies. This ongoing verification ensures the reliability, timeliness, and accuracy of the deployed alphas.

3.3 Alpha Computation Acceleration Layer

Alpha computation requires preprocessing financial data from various sources such as price-volume data, financial statement data, etc. Because of the computational overhead of processing high-frequency data such as orderbook level data, computational acceleration plays a key role. Also, in alpha mining, billions of alphas are calculated, making the speed of alpha calculations crucial for effectively exploring the alpha search space. Below we outline a few key computation acceleration techniques employed. **Streaming Algorithms**: particularly beneficial in performing rolling window computations on large time-series datasets. They incrementally update calculations with each new data point in the window, significantly optimizing computational efficiency and memory usage for these sliding-scale operations such as `ts_corr`(rolling correlation of time-series data). **Vectorized Computation**: enables efficient financial data processing by eliminating explicit loops, leveraging hardware capabilities for concurrent processing, and optimizing memory management. **SIMD** (Single Instruction, Multiple Data) and **SIMT** (Single Instruction, Multiple Threads) [19]: allows simultaneous computations on multiple data points, fully exploiting hardware capabilities. **Memory Optimization**: pre-allocation of memory pool, layout optimization, and zero-copy, etc, are employed to minimize performance loss from discontinuous memory access and unnecessary memory allocation. **Data Partitioning**: Divides large-scale data into smaller, manageable chunks for independent processing. **Multithreading**: enables parallel computations on each partition using multiple threads. This approach significantly boosts computational speed when processing large financial datasets. **GPU Acceleration** [20]: employs CUDA cores for parallel processing, transforming CPU-bound operations into GPU-accelerated tasks and improves performance for data-intensive computations.

4 EXPERIMENTS

We conduct experiments on Alpha-GPT with the goal of verifying the following research questions (RQ):

- **RQ1**: Can Alpha-GPT generate expressions that are consistent with the input trading ideas?
- **RQ2**: How effective is the algorithmic alpha mining layer in enhancing the seed alphas from LLM?
- **RQ3**: Can users effectively interact with Alpha-GPT to guide the mining process?

Table 1: Comparison of average top-20 out-of-sample IC between alphas generated by Alpha-GPT before and after search enhancement.

	Trend Discrepancy	Shape	RSI	Momentum	Mean Reversion	Flow of Funds
Before search enhancement	0.01151	0.00995	0.01109	0.00951	0.01130	0.00952
After search enhancement	0.02256	0.02190	0.02527	0.02763	0.02187	0.02160



Figure 5: Trading patterns and the corresponding alphas generated by Alpha-GPT that capture them.

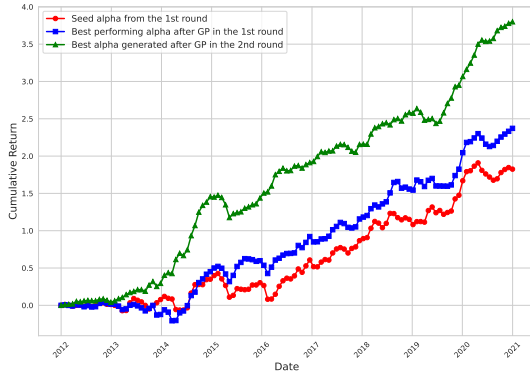


Figure 6: Backtest net values of alphas after different stages of interactions.

- **RQ4:** Can Alpha-GPT successfully explain the trading ideas behind alpha expressions?

4.1 Experimental Setup

The specifications of Alpha-GPT and other relevant information about our experiments include:

Data and operators: We use inter-day volume-price data of stock markets. This data includes the basic candlestick chart data (OHLCV), volume-weighted average price (VWAP), and sector data. We also include 19 basic operators implemented in [21] including time-series operations, cross-sectional operations, group-wise operations and basic element-wise operations.

Knowledge Library: We construct the knowledge library based on the alphas proposed in [2]. For each alpha, we first decompose it into sub-expressions and explain them. Then we explain the combination of these sub-expressions to form the whole trading idea.

Trading idea: Flow of Funds

Alpha: `div(cwise_mul(cwise_max(minus(close, shift(close, 1)), 0), amount, cwise_mul(close, volume)))`

Explanation: This expression calculates the ratio between the inflow of funds on days where the price increased and the product of the closing price and the trading volume of the current day. This ratio represents the intensity of the inflow of funds relative to the trading value.

Trading idea: Volume-price correlation

Alpha: `zscore_scale(ts_corr(close, volume, 20))`

Explanation: This alpha expression aims to capture the correlation between the daily close prices and the trading volume. A high positive correlation may indicate increased trading activity and potentially signal a price trend. The z-score scaling further helps identify extreme or outlier correlation values.

Trading idea: Shadow

Alpha: `div(cwise_max(minus(high, open), minus(high, close)), minus(high, low))`

Explanation: The trading idea behind this alpha expression is to identify situations where the upper shadow of a candlestick is relatively large compared to the entire range of the candlestick. This can indicate potential selling pressure or resistance at higher price levels.

Trading idea: Momentum

Alpha: `ts_delta(ts_rank(div(ts_delta(close, 1), close), 10), 1)`

Explanation: This alpha expression calculates the time series delta between the current value and the previous value of the rank of the ratio between the time series delta of the daily closing price and the current closing price over a period of 10 days. The trading idea is to capture the change in the ranking of stocks based on the relative price changes, which can provide insights into shifts in market sentiment or price momentum.

Figure 7: Alphas generated based on trading ideas and the corresponding explanations generated by Alpha-GPT.

Document embeddings are indexed via Faiss¹. Note that we only employed external memory when generating alphas for trading ideas that align well with those in [2].

LLM and Adapter: For natural language interaction, we use OpenAI’s chat completion API with “gpt-3.5-turbo-16k-0613” model base. For the embedding model used in knowledge retrieval, we use OpenAI’s “text-ada-embedding-002” API with embedding dimension of 1536. The LLM generates a batch of alphas at a time, and will be asked to correct alphas with syntax or semantic errors.

Alpha searching and evaluation: Alphas are searched by the genetic programming model with a fitness score defined by the Information Coefficient (IC). We evaluate the performance of those alphas on out-of-sample criteria such as IC, annual return, Sharpe ratio, etc.

¹<https://github.com/facebookresearch/faiss>

4.2 Idea-Formula Consistency

We first demonstrate that Alpha-GPT can generate formulaic alphas that are consistent with the user’s given trading idea. Figure 5 illustrates three alpha expressions generated based on given trading ideas and their correspondence to the patterns in the candlestick chart. The candlestick chart is plotted from the weekly data of the S&P500 index from 2020 to 2023.

The first trading idea aims to capture golden cross patterns. The alpha value reflects the divergence between the fast and slow moving average curves. The second trading idea characterizes the breakout signals of Bollinger bands, and the corresponding alpha is a binary signal that gets activated when the upper bound is crossed. The third trading idea aims to capture three consecutive bullish movements on the candlestick chart, and the generated alpha successfully identified those patterns. These examples demonstrate that the generated alphas correctly capture the trading ideas.

4.3 Search Enhancement

Table 1 compares the out-of-sample IC of alphas before and after search enhancement by the algorithmic alpha mining layer on 7 different trading ideas. We can see that search enhancement significantly improves the performance of Alpha-GPT and is critical in the Alpha-GPT workflow.

4.4 Human-AI Interaction

Figure 6 illustrates the backtest curve of the alpha generated throughout the human-AI interaction. The backtest is conducted on US stock data from 2012 to 2021. The initial trading idea is to characterize the long-short imbalance. After several rounds of search enhancement and user interaction, the backtest performance of the resulting alphas significantly improved. More details about these interactions are present in the UI example in Figure 3.

4.5 Alpha Explanation

Figure 7 presents examples of alpha expressions generated by Alpha-GPT based on given trading ideas, and the corresponding natural language explanations of these alphas also generated by Alpha-GPT. From these examples we can see that Alpha-GPT can provide appropriate explanations of the generated alphas, relieving the burden of human researchers to interpret these expressions by themselves.

5 RELATED WORK

A lot of algorithms have been studied for formulaic alpha mining. Examples include Monte Carlo random search, Markov-chain Monte Carlo [22], genetic programming [23] and their variants [3], and reinforcement learning [24]. However, these methods all require the user to directly define the algorithmic configurations, providing limited interactivity compared with Alpha-GPT. Meanwhile, LLMs such as GPT [25] have demonstrated emergent abilities [26] and achieved superior performance on various tasks. Besides, LLMs have also shown great reasoning [27, 28] and planning capabilities [29]. In this way, an LLM can be regarded as a core thinking module and be integrated with various peripheral tools [30] to form intelligent LLM-powered agents [31].

6 CONCLUSION AND FUTURE WORK

In this paper, we propose a new paradigm and a new system for alpha mining by leveraging the power of large language models. Further study on prompt engineering, LLM fine-tuning, alpha search algorithms, and knowledge library construction can be conducted to improve the capability of this system.

REFERENCES

- [1] Igor Tulchinsky. Introduction to Alpha Design. In *Finding Alphas*, pages 1–6. John Wiley & Sons, Ltd, 2019.
- [2] Zura Kakushadze. 101 Formulaic Alphas. *arXiv:1601.00991 [q-fin]*, March 2016. arXiv: 1601.00991.
- [3] Tianping Zhang, Yuanqi Li, Yifei Jin, and Jian Li. AutoAlpha: an Efficient Hierarchical Evolutionary Algorithm for Mining Alpha Factors in Quantitative Investment, April 2020. arXiv:2002.08245 [q-fin].
- [4] Andrew W. Lo, Harry Mamaysky, and Jiang Wang. Foundations of Technical Analysis: Computational Algorithms, Statistical Inference, and Empirical Implementation. *The Journal of Finance*, 55(4):1705–1765, 2000. [_eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/0022-1082.00265](https://onlinelibrary.wiley.com/doi/pdf/10.1111/0022-1082.00265).
- [5] R.N. Elliott and R.R. Prechter. *R.N. Elliott’s Masterworks: The Definitive Collection*. New Classics Library, 2005.
- [6] Jian Guo, Saizhuo Wang, Lionel M. Ni, and Heung-Yeung Shum. Quant 4.0: Engineering Quantitative Investment with Automated, Explainable and Knowledge-driven Artificial Intelligence, December 2022. arXiv:2301.04020 [cs, q-fin].
- [7] slundberg/shap: A game theoretic approach to explain the output of any machine learning model.
- [8] Yufei Wu, Daniele Magazzini, and Manuela Veloso. How Robust are Limit Order Book Representations under Data Perturbation? In *ICML Workshop on Representation Learning for Finance and E-Commerce Applications*, 2021.
- [9] Fully Sharded Data Parallel: faster AI training with fewer GPUs, July 2021.
- [10] Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?, October 2022. arXiv:2202.12837 [cs].
- [11] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing, July 2021. arXiv:2107.13586 [cs].
- [12] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep Reinforcement Learning from Human Preferences. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 4299–4307, 2017.
- [13] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. *arXiv:1910.02054 [cs, stat]*, May 2020. arXiv: 1910.02054.
- [14] Song Han, Huizi Mao, and William J. Dally. Deep Compression: Compressing Deep Neural Network with Pruning, Trained Quantization and Huffman Coding. In Yoshua Bengio and Yann LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.
- [15] Selim Amrouni, Aymeric Moulin, Jared Vann, Svitlana Vyetenrenko, Tucker Balch, and Manuela Veloso. ABIDES-Gym: Gym Environments for Multi-Agent Discrete Event Simulation and Application to Financial Markets. In *Proceedings of the Second ACM International Conference on AI in Finance, ICAIF ’21, New York, NY, USA, 2021*. Association for Computing Machinery.
- [16] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. Similarity Search in High Dimensions via Hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases, VLDB ’99*, pages 518–529, San Francisco, CA, USA, 1999. Morgan Kaufmann Publishers Inc.
- [17] Scott M Lundberg and Su-In Lee. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [18] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. “Why Should I Trust You?”: Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD ’16*, pages 1135–1144, New York, NY, USA, 2016. Association for Computing Machinery.
- [19] John L. Hennessy and David A. Patterson. *Computer Architecture, Fifth Edition: A Quantitative Approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 5th edition, 2011.
- [20] NVIDIA Corporation. NVIDIA CUDA C Programming Guide, 2010.
- [21] Jiadong Guo, Jingshu Peng, Hang Yuan, and Lionel Ming-shuan Ni. HXPY: A High-Performance Data Processing Package for Financial Time-Series Data. *Journal of Computer Science and Technology*, 38(1):3–24, February 2023.

- [22] Ying Jin, Weilin Fu, Jian Kang, Jiadong Guo, and Jian Guo. Bayesian Symbolic Regression, January 2020. arXiv:1910.08892 [stat].
- [23] Can Cui, Wei Wang, Meihui Zhang, Gang Chen, Zhaojing Luo, and Beng Chin Ooi. AlphaEvolve: A Learning Framework to Discover Novel Alphas in Quantitative Investment. In *Proceedings of the 2021 International Conference on Management of Data*, pages 2208–2216, Virtual Event China, June 2021. ACM.
- [24] Shuo Yu, Hongyan Xue, Xiang Ao, Feiyang Pan, Jia He, Dandan Tu, and Qing He. Generating Synergistic Formulaic Alpha Collections via Reinforcement Learning, May 2023. arXiv:2306.12964 [cs, q-fin].
- [25] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. *arXiv:2005.14165 [cs]*, July 2020. arXiv: 2005.14165.
- [26] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent Abilities of Large Language Models. *Transactions on Machine Learning Research*, June 2022.
- [27] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. May 2022.
- [28] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models, May 2023. arXiv:2305.10601 [cs].
- [29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. September 2022.
- [30] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools, February 2023. arXiv:2302.04761 [cs].
- [31] Lilian Weng. LLM Powered Autonomous Agents, June 2023. Section: posts.